

AI Operations (AIOps) — Module 3 Assignment

Infrastructure & Containerization

Name: Saketh Pranav

Roll No.: DA24B042

Repository: https://github.com/SakethPranav/AIOps_Assignment_2

Q1 — Single-Stage vs. Multi-Stage Docker

Both images return HTTP 200 on `/healthz`, `spam` for the spam test message and `ham` for the ham test message.

Image	Disk usage	Content size
Naive (single-stage, <code>python:3.11</code>)	2.15 GB	572 MB
Multi-stage (<code>python:3.11-slim</code> runtime)	625 MB	143 MB
Reduction	70.9%	75.0%

Why it is smaller. The naive image is built on the full `python:3.11`, which includes gcc, make and development headers, and its `/app` also keeps `spam_dataset.csv`, `train.py`, `generate_dataset.py`, `Dockerfile.naive` and `__pycache__`. The multi-stage image does the install and training in a builder stage, and its final stage starts from `python:3.11-slim` and copies in only `/opt/venv`, `model.joblib` and `app.py`, so the compilers, dataset, training scripts and builder layers are left behind.

Q2 — Docker Compose with Redis Cache

Implementation. On each `/predict` call the API looks up a Redis key made of the prefix `spam:` and the SHA-256 hash of the input text. On a hit it returns the cached label; on a miss it runs the model, stores the label with a 300s TTL and returns it. The response body is unchanged (`{"label": ...}`); hit or miss is reported in the `X-Cache` header and handler time in `X-Handler-Time-Ms`.

Compose stack. `docker compose ps` shows both services Up: `api` (`spam-api:redis`, port 8000 published) and `cache` (`redis:7-alpine`). The API reaches Redis at the hostname `cache` through `REDIS_HOST=cache`.

Correctness on miss and hit. Each message was sent twice after `FLUSHALL`, and the label was identical on both calls. `redis-cli keys 'spam:*'` listed two keys, and `ttd` on one of them returned 296s.

Message	Label (miss and hit)	Miss handler	Hit handler
spam test message	spam	7.270 ms	0.507 ms
ham test message	ham	5.336 ms	0.531 ms

Cache-hit speedup. `timing.py` sent 30 unique messages, each once (miss) and then three more times (hits), after a warm-up request. It asserts that every repeat is a hit with the same label as the miss, and it completed for all 30 messages.

Metric	Miss ($n = 30$)	Hit ($n = 90$)	Difference
Handler time (avg)	3.104 ms	0.812 ms	73.8% lower
End-to-end time (avg)	7.92 ms	5.59 ms	29.4% lower

Excerpt from `docker compose logs api` (one miss followed by three hits):

```
api-1 | INFO:    cache=MISS handler_ms=4.463
api-1 | INFO:    cache=HIT handler_ms=0.799
api-1 | INFO:    cache=HIT handler_ms=0.943
api-1 | INFO:    cache=HIT handler_ms=1.005
```

Compose vs. Kubernetes. Docker Compose defines several containers, with their shared network, environment and startup order, in one `docker-compose.yml` and starts them with one command, which a single Dockerfile cannot do; here it lets the API reach Redis as `cache`. Compose runs containers on a single host, whereas Kubernetes orchestrates containers across a cluster of nodes with scheduling, self-healing, scaling and rolling updates.